SEMANA 03 - SESION 1

DDD tactico y arquitectura hexagonal

El corazon de un sistema distribuido mantenible

Sistemas Distribuidos (cod. 82739) - 2026-B, Grupo 2 - CORHUILA

Idea central de la sesion

"Si tu agregado necesita la base de datos para hacer cumplir una regla, la regla esta en el lugar equivocado."

Objetivos de la sesion

1) Modelar con DDD tactico (entidad, VO, agregado, evento). 2) Aplicar hexagonal (puertos/adaptadores, regla de dependencia). 3) Mantener el dominio libre de I/O y framework. 4) Reconocer y corregir violaciones de frontera.

Que se trabajo

En un sistema distribuido la complejidad dificil es **el negocio, no la plomeria**. DDD pone las reglas en una capa de dominio pura con **lenguaje ubicuo** compartido. Todo lo demas -HTTP, BD, colas- es detalle que no puede filtrarse al nucleo.

El hexagono - las flechas apuntan hacia adentro

`ENTRADA (controlador HTTP, cola) -> NUCLEO (dominio + casos de uso, sin I/O) -> SALIDA (BD, API externa)`

adaptadores -> aplicacion -> dominio, nunca al reves.

Las cuatro piezas del DDD tactico

Pieza	Que es
Value Object	Sin identidad; igualdad por valor; inmutable (Money, Address)
Entidad	Identidad que persiste en el cambio (Order, Customer)
Agregado (raiz)	El UNICO lugar donde cambian los invariantes; se guarda entero via repositorio (puerto)
Evento de dominio	Algo que ya paso: inmutable, en pasado (OrderConfirmed)

La regla que mantiene todo limpio

El dominio no importa NADA de infraestructura. Si tu `Order` necesita la BD para hacer cumplir una regla, la regla esta en el lugar equivocado. Los repositorios son interfaces en el dominio, implementadas en infraestructura.

Ejemplo aplicado: el modelo anemico

Un equipo entrega `Order` como getters/setters publicos y mete las reglas en el servicio. Seis semanas despues la misma regla esta duplicada en 3 lugares y una copia esta mal.

DDD tactico y arquitectura hexagonal

El corazon de un sistema distribuido mantenible

Antes (anemico) vs despues (rico)

```
// ANTES: la regla vive fuera del objeto
class Order { public List<Line> lines; public String status; }
if (o.lines.isEmpty()) throw ...; // se repite en 3 sitios

// DESPUES: el invariante vive DENTRO del agregado
class Order {
  private final List<Line> lines; // privado
  void confirm(){
    if (lines.isEmpty()) throw new DomainException(...); // UN lugar
    status = Status.CONFIRMED;
  }
}
```

Errores comunes a evitar

- **Dominio anemico:** datos sin comportamiento, reglas dispersas.
- Tipos de infraestructura filtrandose a `domain/`.
- Agregados **demasiado grandes** o **demasiado pequenos**.
- Mutar un hijo **dentro** de un agregado sin pasar por la raiz.

Para esta semana

Elige un contexto acotado de tu producto y modelalo: raiz del agregado, entidades/VOs, invariantes que protege, eventos que emite.

Distributed Systems - Semana 3, Sesion 1 - Faculty of Engineering, CORHUILA 2026-B. Material: code-corhuila.github.io/ova-web/2026-B/distribuidos/